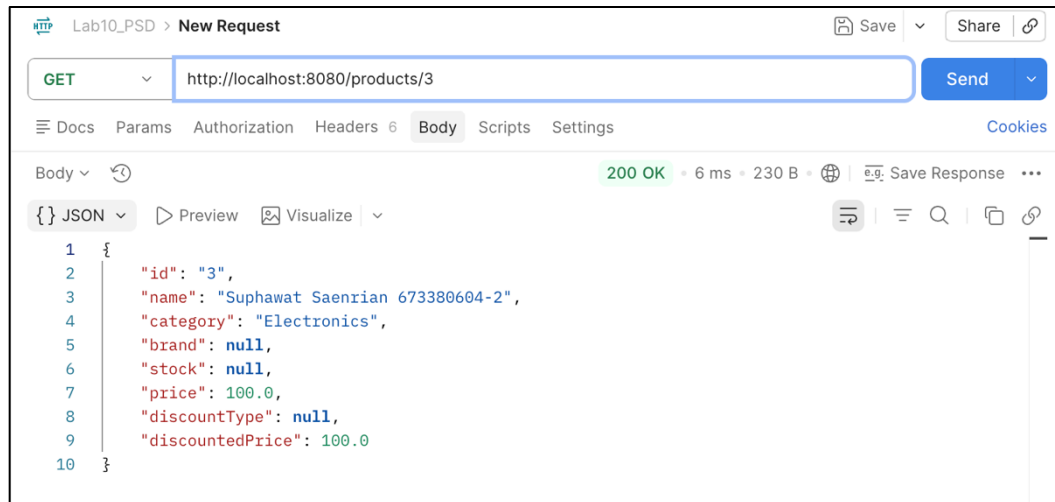


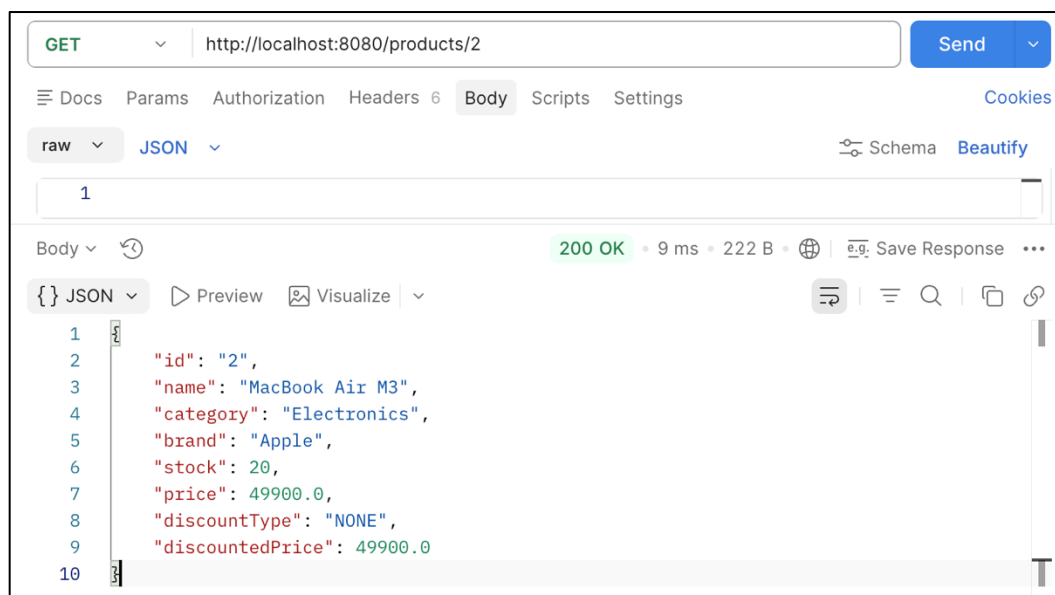
Lab 10 WebClient Report

ผลลัพธ์ใน response มีชื่อและรหัสนักศึกษา (ใน Product name ที่ seed ไว้)

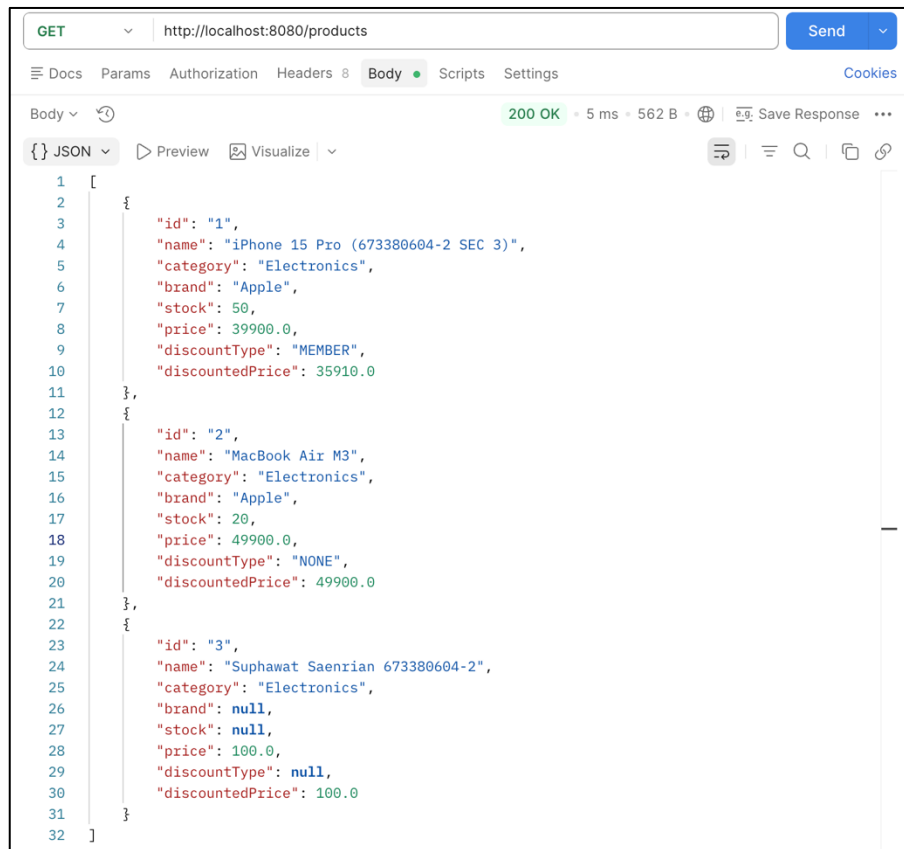


End point

1. GET /products/{id} → Mono<Product>



2. GET /products → Flux<Product>



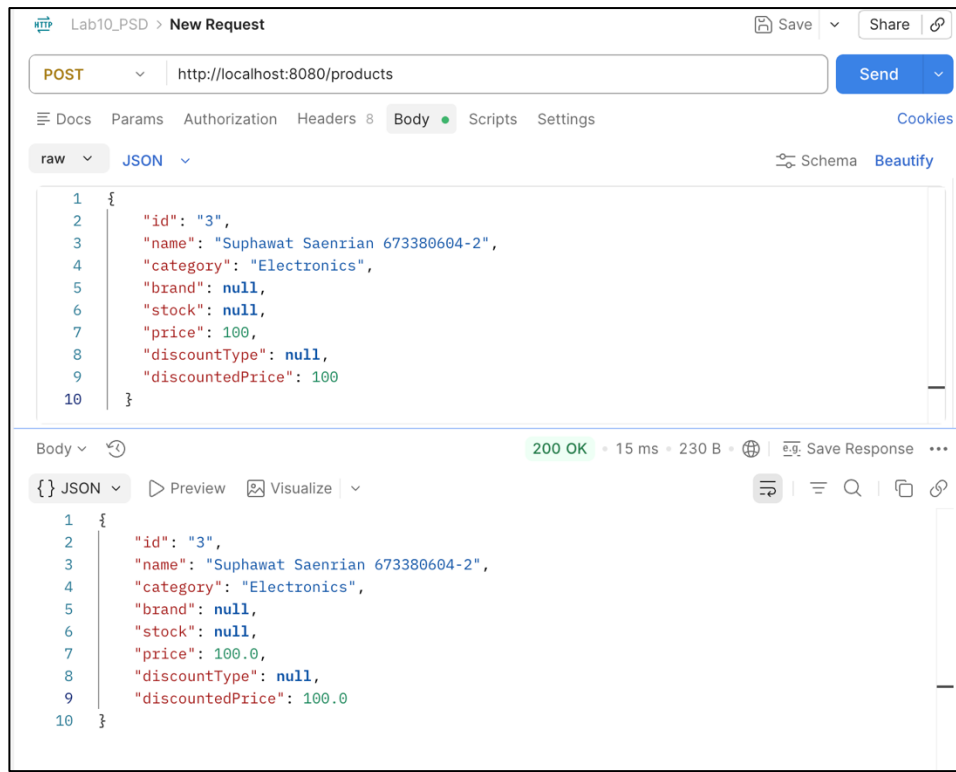
The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://localhost:8080/products
- Status:** 200 OK
- Time:** 5 ms
- Size:** 562 B
- Body Type:** JSON

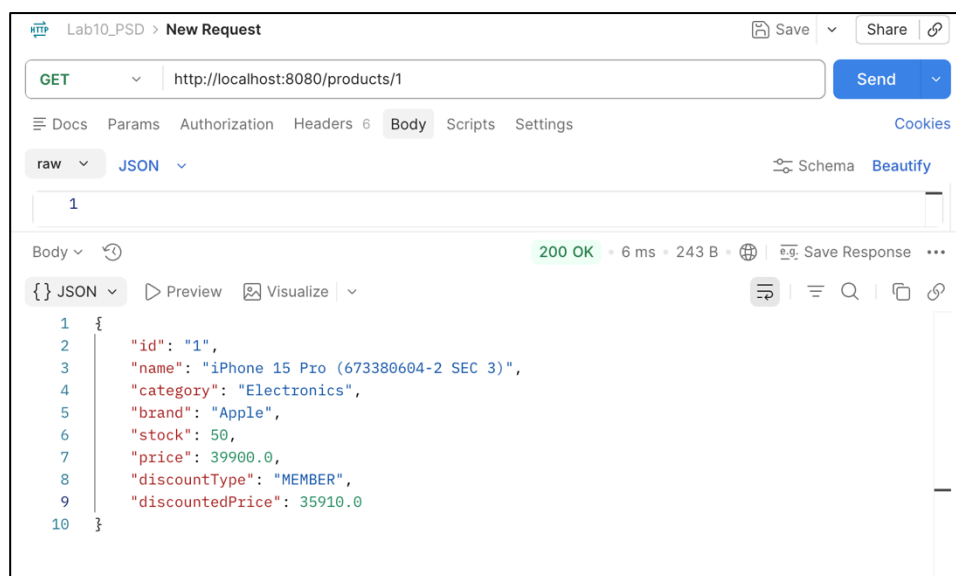
The JSON response is as follows:

```
[
  {
    "id": "1",
    "name": "iPhone 15 Pro (673380604-2 SEC 3)",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 50,
    "price": 39900.0,
    "discountType": "MEMBER",
    "discountedPrice": 35910.0
  },
  {
    "id": "2",
    "name": "MacBook Air M3",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 20,
    "price": 49900.0,
    "discountType": "NONE",
    "discountedPrice": 49900.0
  },
  {
    "id": "3",
    "name": "Supawat Saenrian 673380604-2",
    "category": "Electronics",
    "brand": null,
    "stock": null,
    "price": 100.0,
    "discountType": null,
    "discountedPrice": 100.0
  }
]
```

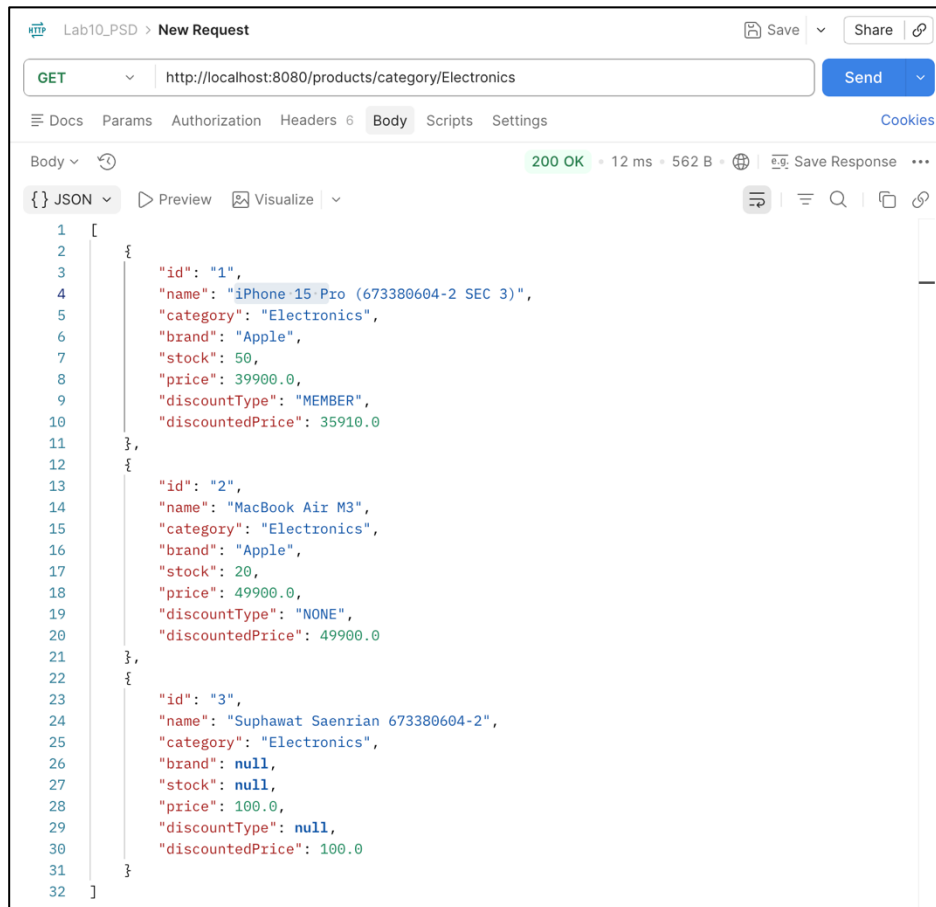
3. POST /products → Mono<Product>



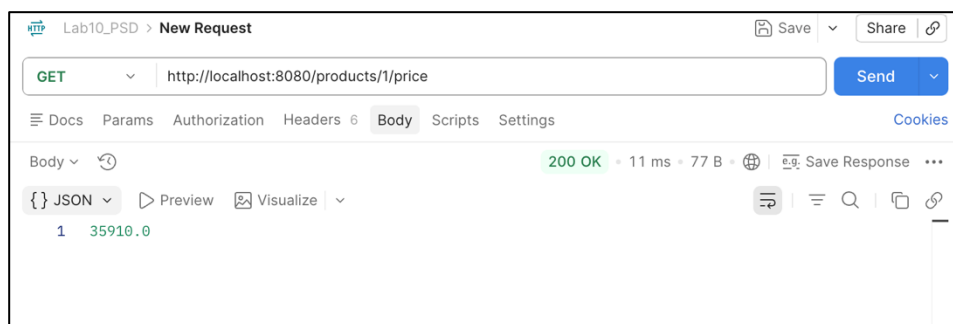
4. DELETE /products/{id} → Mono<Void>



5. GET /products/category/{cat} → Flux<Product>



6. GET /products/{id}/price → Mono<Double>



1. Reactive Programming vs Blocking ต่างกันอย่างไร

ตอบ Blocking I/O คือรูปแบบการทำงานที่ Thread ต้องหยุดรอผลลัพธ์จากการอ่านหรือเขียนข้อมูล เช่น การเรียก API หรือ การติดต่อ Database จนกว่าจะได้รับผลลัพธ์จึงสามารถทำงานต่อได้ หากมีหลาย Request พร้อมกัน อาจทำให้ Thread ต้องรอเป็นจำนวนมากและใช้ทรัพยากรเพิ่มขึ้น

Reactive Programming คือรูปแบบการเขียนโปรแกรมที่รองรับการทำงานแบบ Asynchronous และ Non-blocking โดยเมื่อมีการรอข้อมูล Thread สามารถไปทำงานอื่นได้ และเมื่อข้อมูลพร้อมจึงส่งผลลัพธ์กลับมา ทำให้เหมาะกับระบบที่มีหลาย Request หรือมีการรับส่งข้อมูลต่อเนื่อง

Blocking I/O	Reactive Programming
Thread ต้องรอผลลัพธ์	Thread ไม่จำเป็นต้องรอ
ใช้ Thread ต่อ Request	สามารถใช้ Thread จำนวนน้อยรองรับหลาย Request
การรอ I/O อาจทำให้ Thread ไม่ได้ทำงาน	ลดการเสียเวลาในการรอ I/O
คืนค่าผลลัพธ์โดยตรง เช่น String	คืนค่าเป็น Mono หรือ Flux
เหมาะกับงานทั่วไปที่มีการประมวลผลแบบปกติ	เหมาะกับงานที่มีการรอข้อมูลและต้องรองรับหลาย Request

2. Mono vs Flux ใช้กรณีไหน และทำไม

ตอบ Mono เป็น Publisher ใน Project Reactor ที่ใช้แทนข้อมูลจำนวน 0 หรือ 1 ค่า โดยสามารถส่งข้อมูลผ่าน onNext() ได้สูงสุด 1 ครั้ง และจบการทำงานด้วย onComplete() หรือ onError() เหมาะกับการทำงานที่คาดว่าจะได้ผลลัพธ์เพียงรายการเดียว

ตัวอย่างการใช้ Mono ใน Lab 10 ได้แก่ findById() สำหรับค้นหา Product ตาม ID, save() สำหรับบันทึก Product และ getDiscountedPrice() สำหรับคำนวณราคาหลังส่วนลด

Flux เป็น Publisher ที่ใช้แทนข้อมูลจำนวน 0 ถึงหลายค่า สามารถส่งข้อมูลออกมาได้หลายรายการ เหมาะกับการดึงข้อมูลทั้งหมดหรือการกรองข้อมูลจากหลายรายการ

ตัวอย่างการใช้ Flux ใน Lab 10 ได้แก่ findAll() สำหรับดึง Product ทั้งหมด และ findByCategory() สำหรับกรอง Product ตาม Category

Mono<T>	Flux<T>
คืนค่า 0 หรือ 1 ค่า	คืนค่า 0 ถึงหลายค่า
Mono<Product>	Flux<Product>

findById()	findAll()
save()	findByCategory()
getDiscountedPrice()	getAll()
เหมาะกับข้อมูล 1 รายการ	เหมาะกับข้อมูลหลายรายการ

3. WebClient อธิบาย Method Chain

ตอบ WebClient เป็นเครื่องมือของ Spring WebFlux ที่ใช้สำหรับเรียก HTTP API แบบ Reactive และ Non-blocking โดยใน Lab 10 ใช้ WebClient เรียก API ของ Product Server ที่ <http://localhost:8080>

คำอธิบาย Method Chain

1. **.get()** กำหนด HTTP Method เป็น GET เพื่อขอข้อมูลจาก Server เช่น การค้นหา Product
2. **.uri()** กำหนด URL ที่ต้องการเรียก โดยสามารถส่งค่า Parameter เช่น ID เข้าไปใน URL ได้
3. **.retrieve()** เริ่มรับ Response จาก Server และเตรียมจัดการข้อมูลที่ได้รับกลับมา
4. **.bodyToMono(Product.class)** แปลงข้อมูล Response Body เป็น Mono<Product> เหมาะกับการรับ Product 1 รายการ
5. **.bodyToFlux(Product.class)** แปลงข้อมูล Response Body เป็น Flux<Product> เหมาะกับการรับ Product หลายรายการ

ตัวอย่างการใช้ WebClient แบบ GET หลายรายการ

```
public Flux<Product> getAllProducts() {  
    return client.get()  
        .uri("/products")  
        .retrieve()  
        .bodyToFlux(Product.class);  
}
```

Method นี้ใช้ GET เพื่อเรียก Product ทั้งหมดจาก Server และแปลง Response เป็น Flux<Product> เนื่องจากผลลัพธ์อาจมีหลายรายการ

ตัวอย่างการใช้ WebClient แบบ POST

```
public Mono<Product> createProduct(Product product) {
```

```

return client.post()
    .uri("/products")
    .bodyValue(product)
    .retrieve()
    .bodyToMono(Product.class);
}

```

- .post() กำหนด HTTP Method เป็น POST เพื่อส่งข้อมูลไปยัง Server
- .uri("/products") กำหนด URL สำหรับเพิ่ม Product
- .bodyValue(product) กำหนดข้อมูล Product ที่ต้องการส่งใน Request Body
- .retrieve() รับ Response จาก Server
- .bodyToMono(Product.class) แปลง Response เป็น Mono<Product>

4. Code ของ Repository, Service, Controller พร้อม comment อธิบาย

- Repository

```

/**
 * นายศุภวัฒน์ แสนเรียน รหัสนักศึกษา 673380604-2 Section 3
 */
package com.example.lab10.repository;
import com.example.lab10.model.Product;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;

import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

public class ProductRepository {
    private final Map<String, Product> store = new ConcurrentHashMap<>();

    public ProductRepository() {
        store.put("1", new Product("1", "iPhone 15 Pro (67XXXXXXX-X SEC 1)",
            "Electronics", "Apple", 50, 39900.0, "MEMBER"));
        store.put("2", new Product("2", "MacBook Air M3",
            "Electronics", "Apple", 20, 49900.0, "NONE"));
        store.put("3", new Product("3", "Samsung Galaxy S24",
            "Electronics", "Samsung", 30, 29900.0, "SEASONAL"));
    }

    public Mono<Product> findById(String id) {
        Product product = store.get(id);
        if (product == null) {
            return Mono.empty();
        }
        return Mono.just(product);
    }

    public Flux<Product> findAll() {
        return Flux.fromIterable(store.values());
    }

    public Mono<Product> save(Product product) {
        store.put(product.getId(), product);
        return Mono.just(product);
    }
}

```

```

    public Mono<Void> deleteById(String id) {
        store.remove(id);
        return Mono.empty();
    }

    public Flux<Product> findByCategory(String category) {
        return findAll()
            .filter(p -> p.getCategory().equalsIgnoreCase(category));
    }
}

```

1. ProductRepository.java

ProductRepository ทำหน้าที่จัดเก็บและจัดการข้อมูล Product ในหน่วยความจำ โดยใช้ ConcurrentHashMap เป็น In-memory storage จึงไม่จำเป็นต้องเชื่อมต่อ Database

```
private final Map<String, Product> store = new ConcurrentHashMap<>();
```

คำอธิบาย Method

- findById() ค้นหา Product จาก ID ใน store หากพบจะคืนค่าเป็น Mono<Product> แต่ถ้าไม่พบจะคืน Mono.empty()
- findAll() ดึง Product ทั้งหมดจาก store โดยใช้ Flux.fromIterable() เพื่อแปลง Collection เป็น Flux
- save() บันทึก Product ลงใน store โดยใช้ ID เป็น Key และคืนค่า Product ที่บันทึก
- deleteById() ลบ Product จาก store ตาม ID และคืนค่าเป็น Mono<Void>
- findByCategory() เรียก findAll() แล้วใช้ filter() เพื่อเลือกเฉพาะ Product ที่มี Category ตรงกัน

• Service

```

/**
 * นายศุภวัฒน์ แสนเรียน รหัสนักศึกษา 673380604-2 Section3
 */
package com.example.lab10.service;
import com.example.lab10.model.Product;
import com.example.lab10.repository.ProductRepository;
import org.springframework.stereotype.Service;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;
import java.util.UUID;
@Service
public class ProductService {

    private final ProductRepository repository;

    public ProductService(ProductRepository repository) {
        this.repository = repository;
    }
}

```

```
• }
• public Mono<Product> getById(String id) {
•     return repository.findById(id)
•         .switchIfEmpty(
•             Mono.error(new RuntimeException("Product not found: " + id))
•         );
• }
• public Flux<Product> getAll() {
•     return repository.findAll();
• }
• public Mono<Product> save(Product product) {
•     if (product.getId() == null) {
•         product.setId(UUID.randomUUID().toString());
•     }
•
•     return repository.save(product);
• }
• public Mono<Void> delete(String id) {
•     return repository.deleteById(id);
• }
• public Flux<Product> getByCategory(String category) {
•     return repository.findByCategory(category);
• }
• public Mono<Double> getDiscountedPrice(String id) {
•     return getById(id)
•         .map(p -> p.getDiscountedPrice());
• }
• }
```

2. ProductService.java

ProductService ทำหน้าที่จัดการ Business Logic ของระบบ โดยรับคำขอจาก Controller แล้วเรียกใช้งาน Repository เพื่อจัดการข้อมูล Product

คำอธิบาย Method

- getById() เรียก repository.findById(id) และใช้ switchIfEmpty() เพื่อส่ง Error เมื่อไม่พบ Product
- getAll() เรียก repository.findAll() เพื่อดึงข้อมูล Product ทั้งหมด
- save() ตรวจสอบว่า Product มี ID หรือไม่ หากไม่มีจะสร้าง ID ใหม่ด้วย UUID.randomUUID() แล้วบันทึกข้อมูล
- delete() เรียก repository.deleteById(id) เพื่อลบ Product
- getByCategory() เรียก repository.findByCategory(category) เพื่อกรองข้อมูลตาม Category
- getDiscountedPrice() เรียก getById(id) แล้วใช้ map() เพื่อแปลง Product เป็นราคาหลังส่วนลด

- Controller

```
/**
 * นายศุภวัฒน์ แสนเรียน รหัสนักศึกษา 673380604-2 Section 3
 */
package com.example.lab10.controller;
import com.example.lab10.model.Product;
import com.example.lab10.service.ProductService;
import org.springframework.web.bind.annotation.*;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;
@RestController
@RequestMapping("/products")
public class ProductController {

    // — Constructor Injection (DIP - SOLID) —
    private final ProductService service;

    public ProductController(ProductService service) {
        this.service = service;
    }

    @GetMapping("/{id}")
    public Mono<Product> getById(@PathVariable String id) {
        return service.getById(id);
    }

    @GetMapping
    public Flux<Product> getAll() {
        return service.getAll();
    }

    @PostMapping
    public Mono<Product> save(@RequestBody Product product) {
        return service.save(product);
    }

    @DeleteMapping("/{id}")
    public Mono<Void> delete(@PathVariable String id) {
        return service.delete(id);
    }

    @GetMapping("/category/{category}")
    public Flux<Product> getByCategory(@PathVariable String category) {
        return service.getByCategory(category);
    }

    @GetMapping("/{id}/price")
    public Mono<Double> getDiscountedPrice(@PathVariable String id) {
        return service.getDiscountedPrice(id);
    }
}
```

3. ProductController.java

ProductController ทำหน้าที่รับ HTTP Request จากผู้ใช้งาน และส่งต่อไปยัง Service เพื่อประมวลผล จากนั้นคืนผลลัพธ์กลับในรูปแบบ Mono หรือ Flux

คำอธิบาย Annotation และ Endpoint

- @RestController กำหนดให้ Class เป็น REST API Controller
- @RequestMapping("/products") กำหนด Path หลักของ API

- @GetMapping ใช้รับ Request แบบ GET
- @PostMapping ใช้รับ Request แบบ POST สำหรับเพิ่ม Product
- @DeleteMapping ใช้รับ Request แบบ DELETE สำหรับลบ Product
- @RequestBody รับข้อมูล Product จาก JSON ใน Request Body
- @PathVariable รับค่า ID หรือ Category จาก URL